Metrics, Scalers, ROC-AUC and kNN

Spam Dataset

char_freq_:	char_freq_(	char_freq_[	char_freq_!	char_freq_$	char_freq_#	capital_run_length_average	capital_run_length_longest	capital_run_length_total	label
0	0	0	0.778	0	0	3.756	61	278	1
0	0.132	0	0.372	0.18	0.048	5.114	101	1028	1
0.01	0.143	0	0.276	0.184	0.01	9.821	485	2259	1
0	0.137	0	0.137	0	0	3.537	40	191	1
0	0.135	0	0.135	0	0	3.537	40	191	1
0	0.223	0	0	0	0	3	15	54	1
0	0.054	0	0.164	0.054	0	1.671	4	112	1
0	0.206	0	0	0	0	2.45	11	49	1
0	0.271	0	0.181	0.203	0.022	9.744	445	1257	1
0	0.751	0	0	0	0	2	4	10	0
0	0.124	0	0	0	0.11	4.771	63	1064	0
0	0.182	0.091	0.091	0	0	1.212	5	40	0
0.065	0.261	0	0	0	0	1.114	5	39	0
0	0.24	0	0.24	0	0	1.687	10	27	0
0	0	0	0	0	0	1.426	6	97	0

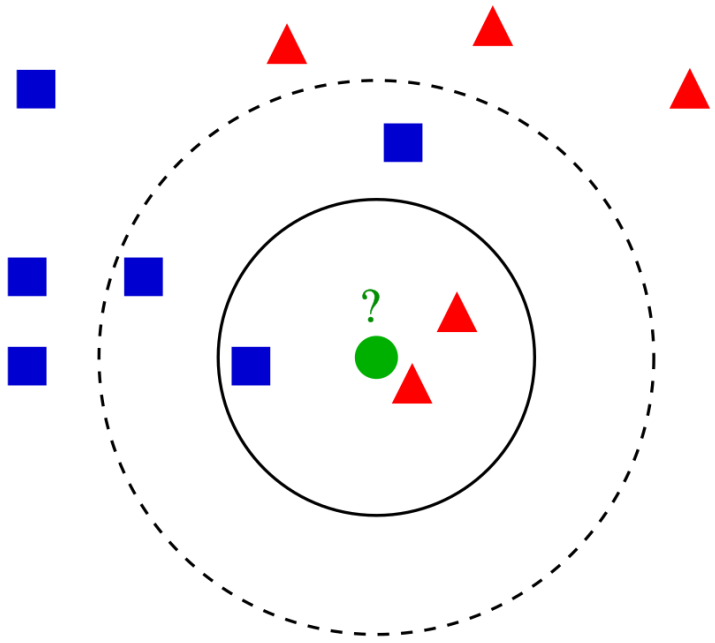
kNN

$$h(x;\ D) = \arg\max_{y \in Y} \sum_{x_i \in D} [y_i = y] w(x_i, x)$$

$w(x_i, x) = 1$, if x_i – one of the k nearest neighbors of x

$w(x_i, x) = 1$, if distance $\rho(x_i, x) < R$ (Radius Neighbors)

$$P(y_i|x) = \frac{\sum_{l=1}^{k} [y_l = y_i]}{k}$$



Classifier Evaluation

		y_i		
		$= y$	$\neq y$	
$h(x_i)$	$= y$	True Positive (TP)	False Positive (FP)	Positive Predictive Value, **Precision** $\frac{TP}{TP + FP}$
	$\neq y$	False Negative (FN)	True Negative (TN)	
		True Positive Rate, **Recall** $\frac{TP}{TP + FN}$	False Positive Rate $\frac{FP}{FP + TN}$	**Accuracy** $\frac{TP + TN}{TP + FP + TN + FN}$

$$F_1 - score = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}} = 2\frac{Precision \times Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

Example 1 (All Negatives)

Positive items: 50 **Negative items: 950**		y_i		
		$= y$	$\neq y$	
$h(x_i)$	$= y$	0 (TP)	0 (FP)	Positive Predictive Value, **Precision** **0**%
	$\neq y$	50 (FN)	950 (TN)	
		True Positive Rate, **Recall** **0**%	False Positive Rate **0**%	**Accuracy** **95**%

$$F_1 - score = 0$$

Example 2 (All Positives)

Positive items: 50 **Negative items: 950**		y_i		
		$= y$	$\neq y$	
$h(x_i)$	$= y$	50 (TP)	950 (FP)	Positive Predictive Value, **Precision** **5%**
	$\neq y$	0 (FN)	0 (TN)	
		True Positive Rate, **Recall** **100%**	False Positive Rate **100%**	**Accuracy** **5%**

$$F_1 - score = 10\%$$

Example 3 (50/50)

Positive items: 50 **Negative items: 950**		y_i		
		$= y$	$\neq y$	
$h(x_i)$	$= y$	25 (TP)	475 (FP)	Positive Predictive Value, **Precision** **5**%
	$\neq y$	25 (FN)	475 (TN)	
		True Positive Rate, **Recall** **50**%	False Positive Rate **50**%	**Accuracy** **50**%

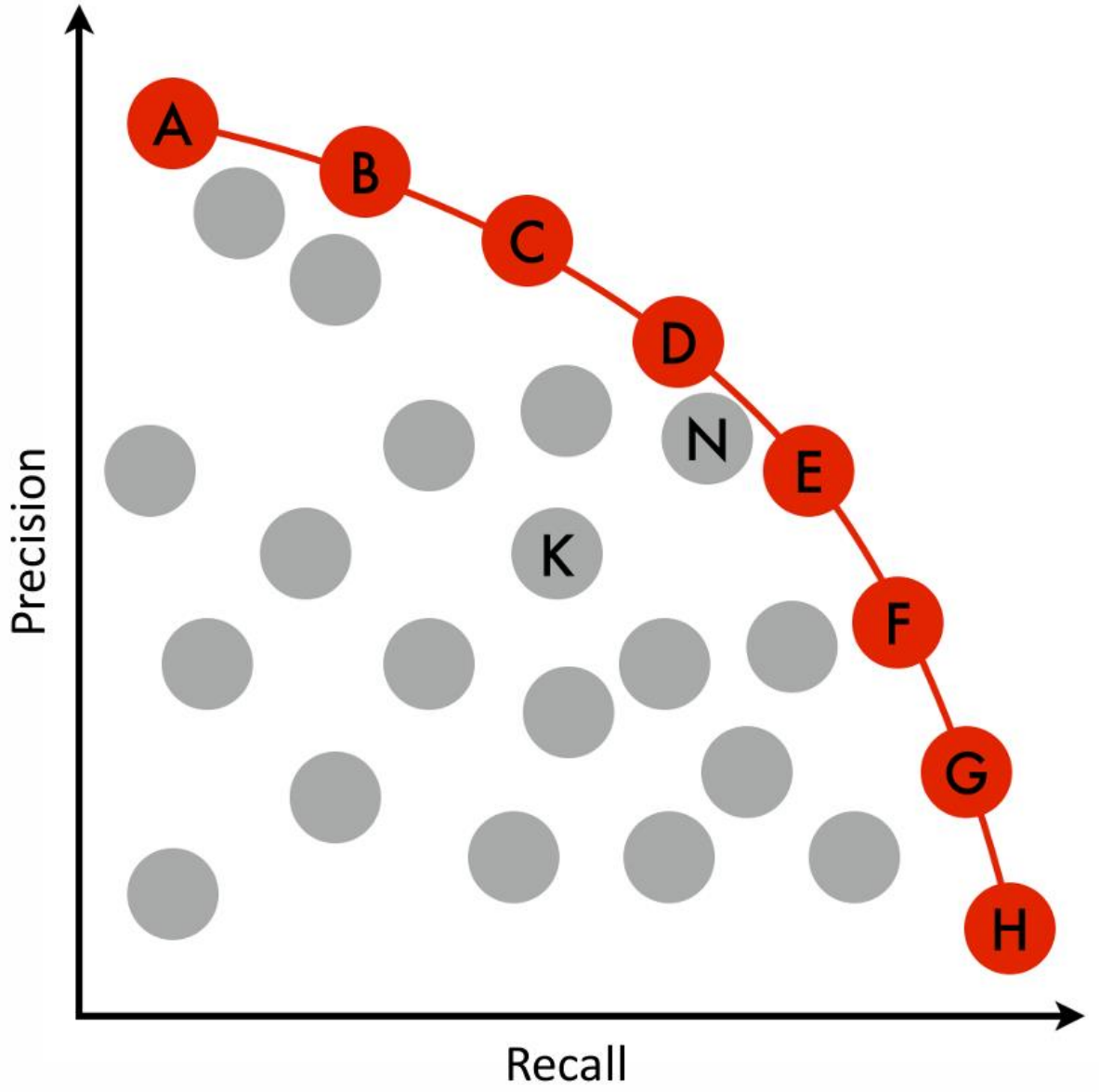
$$F_1 - score = 9\%$$

Example 4 (A Some Classifier)

Positive items: 50 **Negative items: 950**		y_i		
		$= y$	$\neq y$	
$h(x_i)$	$= y$	40 (TP)	100 (FP)	Positive Predictive Value, **Precision** **29**%
	$\neq y$	10 (FN)	850 (TN)	
		True Positive Rate, **Recall** **80**%	False Positive Rate **11**%	**Accuracy** **89**%

$$F_1 - score = 42\%$$

Pareto Efficiency



● – 1st Pareto frontier

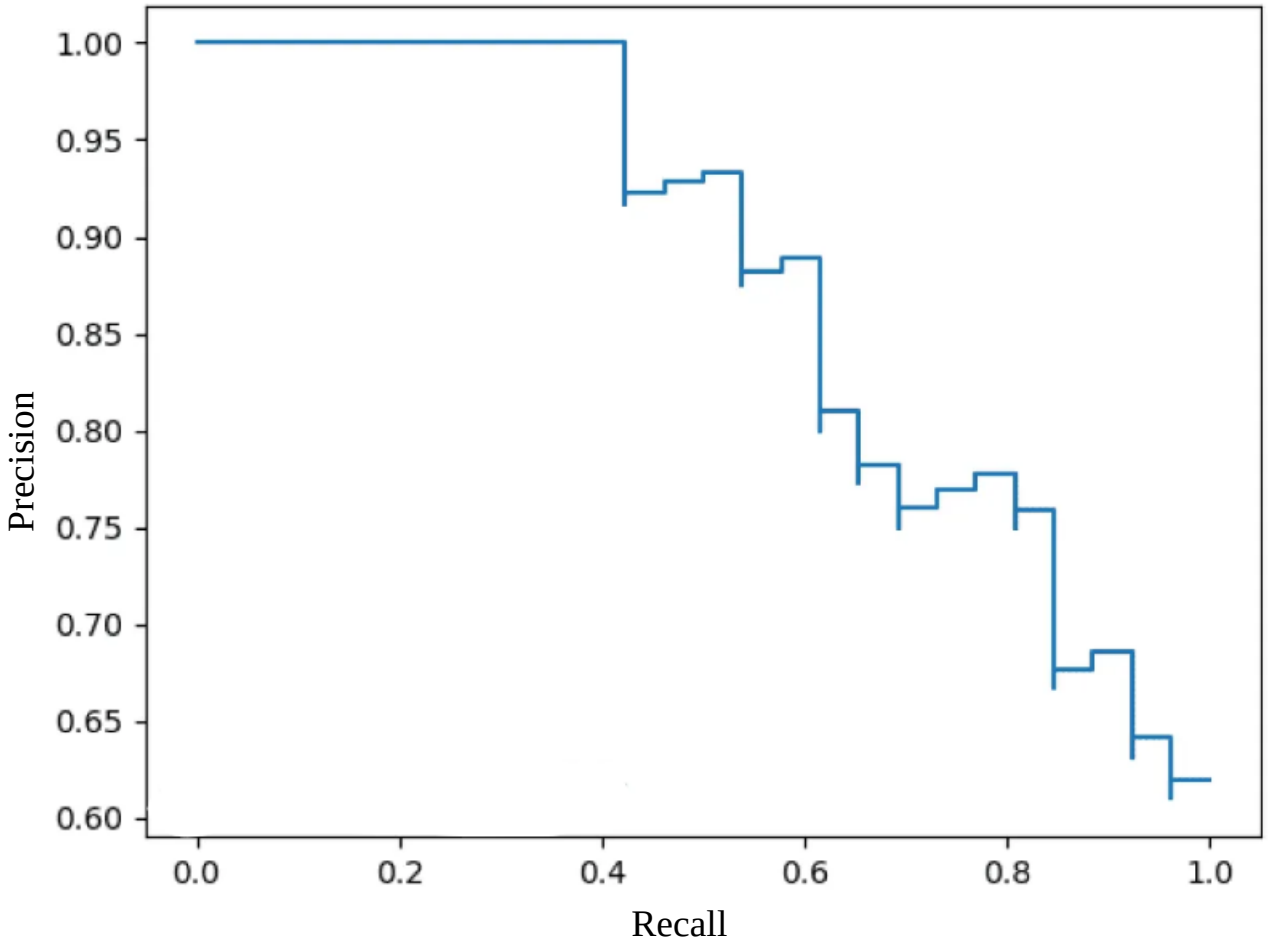
N – 2nd Pareto frontier, K – 3rd

A classifier is Pareto efficient if there is no better classifier in terms of both precision and recall

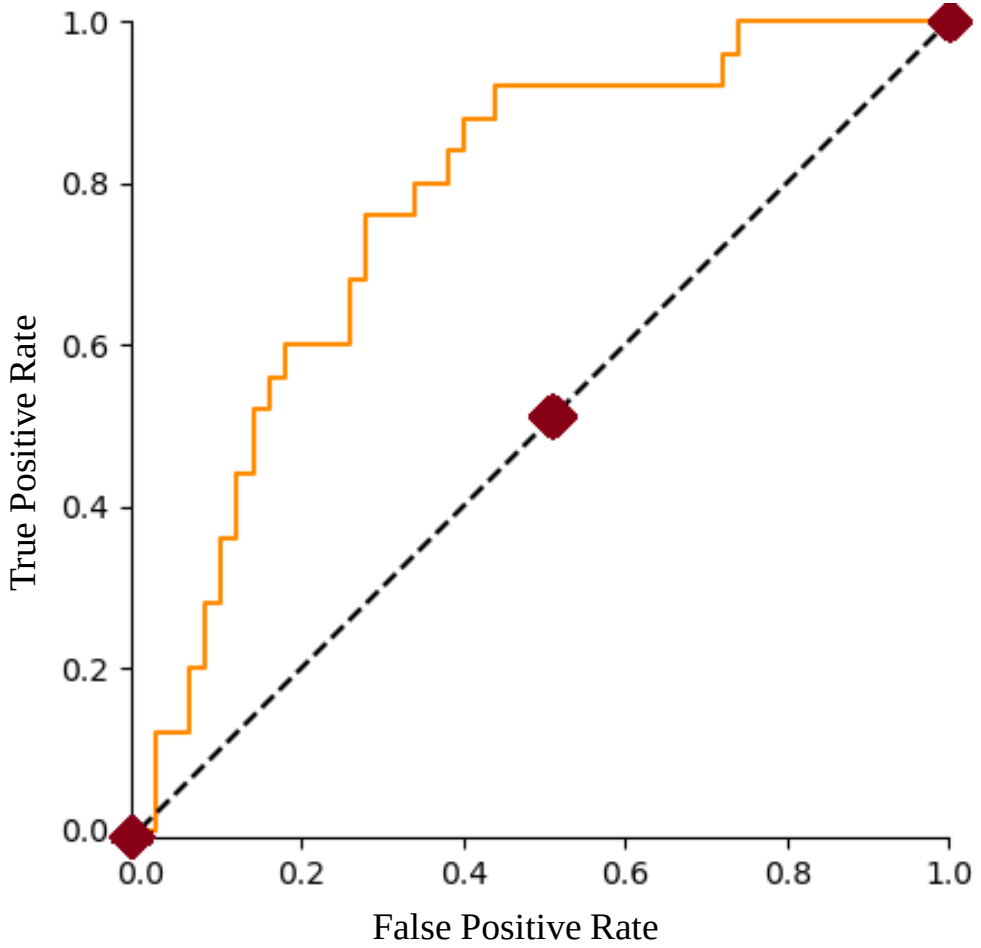
(Instead of precision and recall there can be any other indicators\metrics\etc.)

ROC (**R**eceiver **O**perating **C**haracteristic)

Precision-Recall Curve



ROC Curve



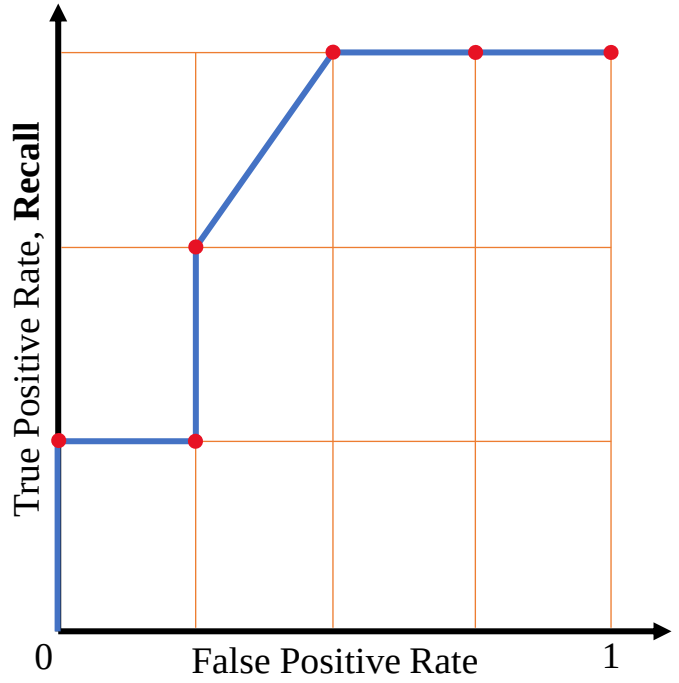
AUC – Area Under the Curve

ROC

Id	Prob	Label
1	0.5	0
2	0.1	0
3	0.2	0
4	0.6	1
5	0.2	1
6	0.3	1
7	0.0	0

Id	Prob	Label
4	0.6	1
1	0.5	0
6	0.3	1
3	0.2	0
5	0.2	1
2	0.1	0
7	0.0	0

Id	> 0.25	Label
4	1	1
1	1	0
6	1	1
3	0	0
5	0	1
2	0	0
7	0	0



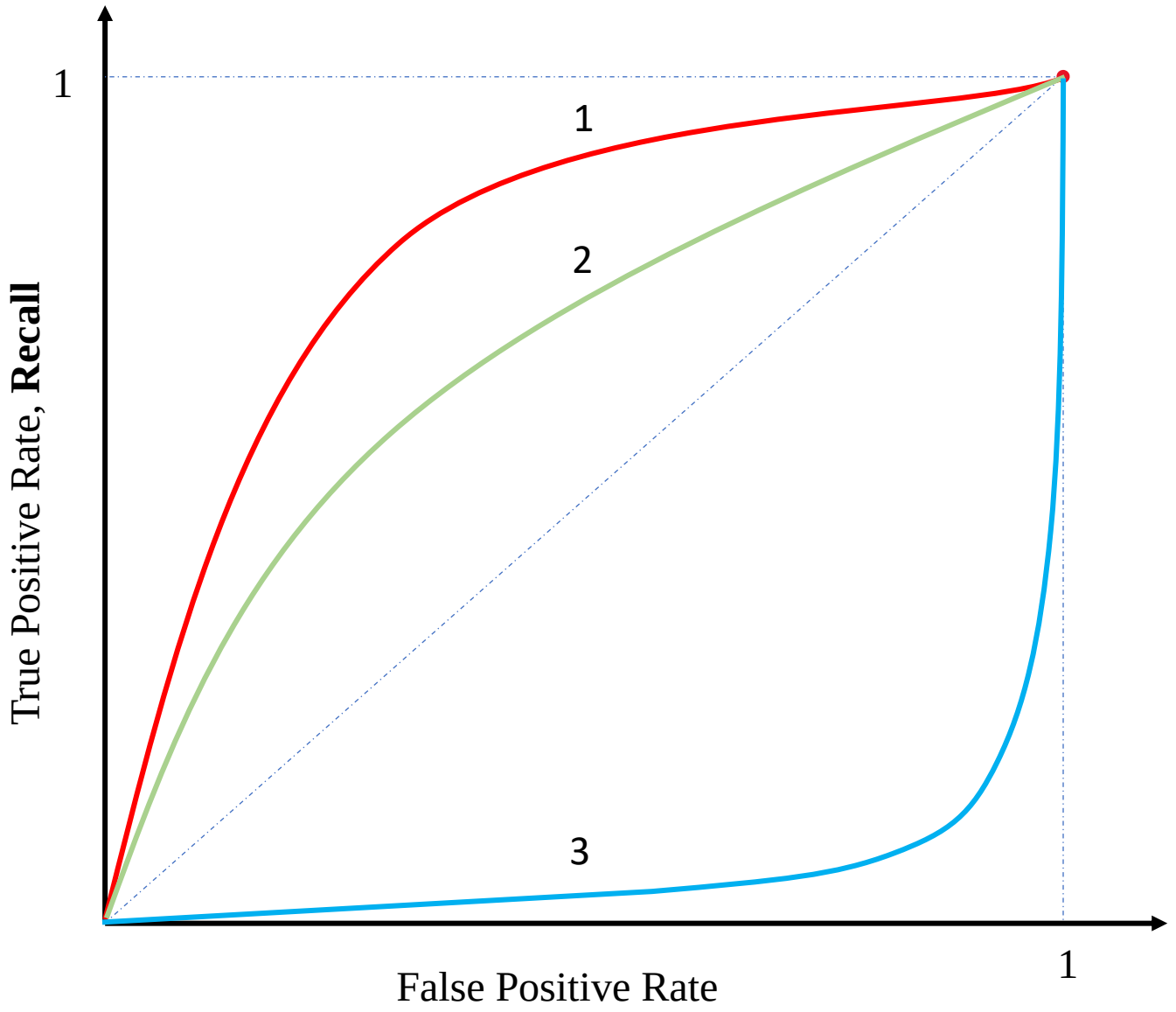
True Positive Rate, **Recall**

$$\frac{TP}{TP + FN} = \frac{TP}{P}$$

False Positive Rate

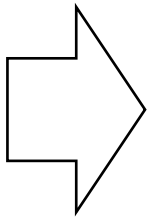
$$\frac{FP}{FP + TN} = \frac{FP}{F}$$

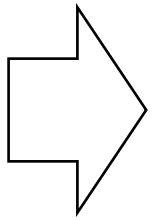
What's Better?



Categorical Features: One Hot Encoding

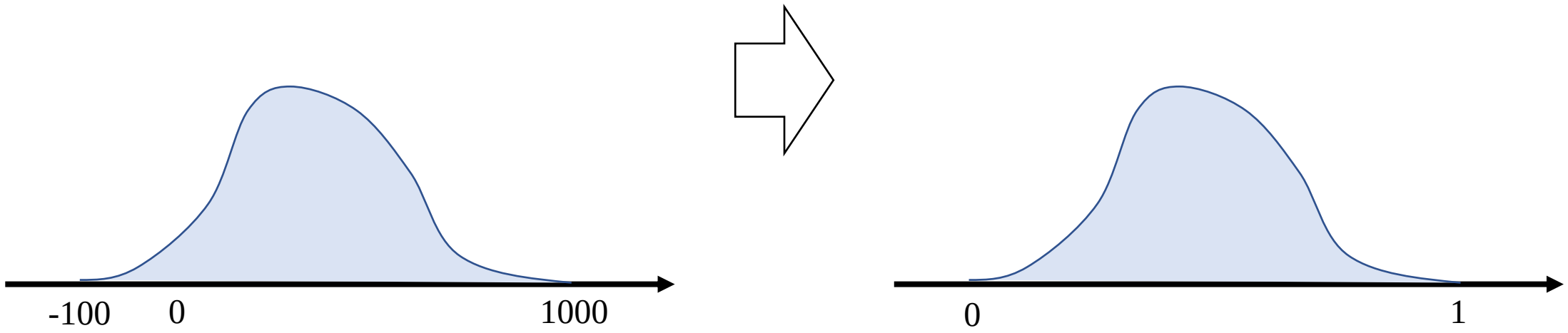
color	color_index	color_red	color_green	color_blue
red	0	1	0	0
green	1	0	1	0
blue	2	0	0	1
red	0	1	0	0





Scalers: MinMax

MinMax scaler: $x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$

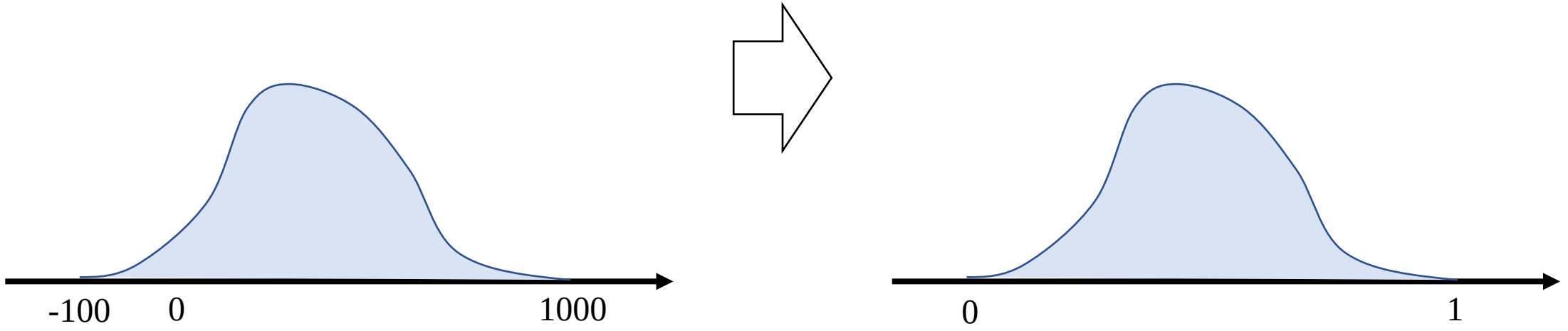


MinMax: How To Use?

$$x_{scaled}^{train} = \frac{x^{train} - x_{min}^{train}}{x_{max}^{train} - x_{min}^{train}}$$

$$x_{scaled}^{\boldsymbol{val}} = \frac{x^{\boldsymbol{val}} - x_{min}^{train}}{x_{max}^{train} - x_{min}^{train}}$$

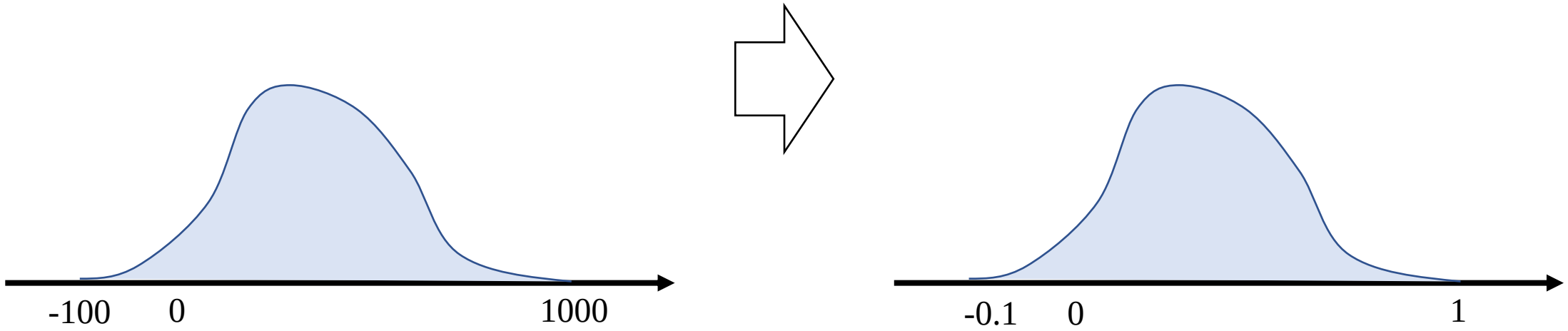
$$x_{scaled}^{\boldsymbol{test}} = \frac{x^{\boldsymbol{test}} - x_{min}^{train}}{x_{max}^{train} - x_{min}^{train}}$$



Scalers: MaxAbs

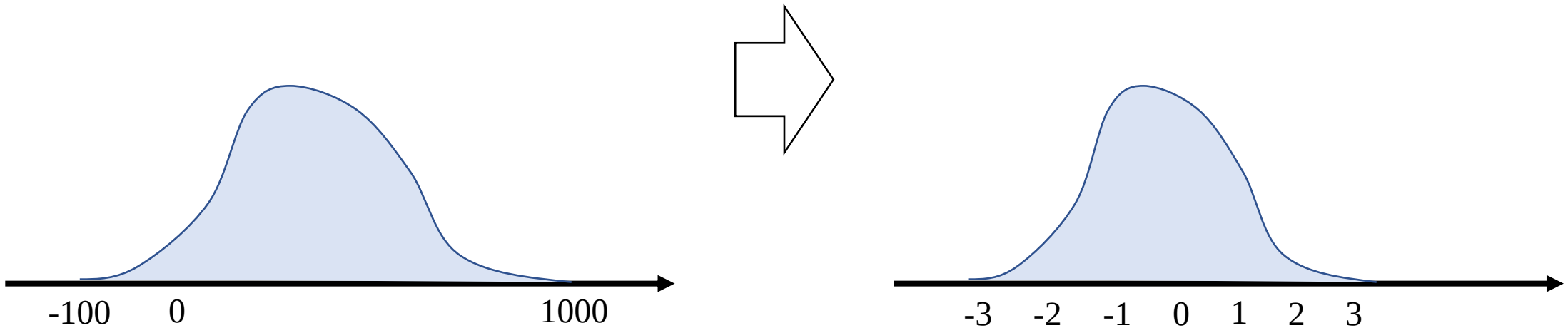
MaxAbs scaler: $x_{scaled} = \frac{x}{|x|_{max}}$

Keep the sparsity of the data

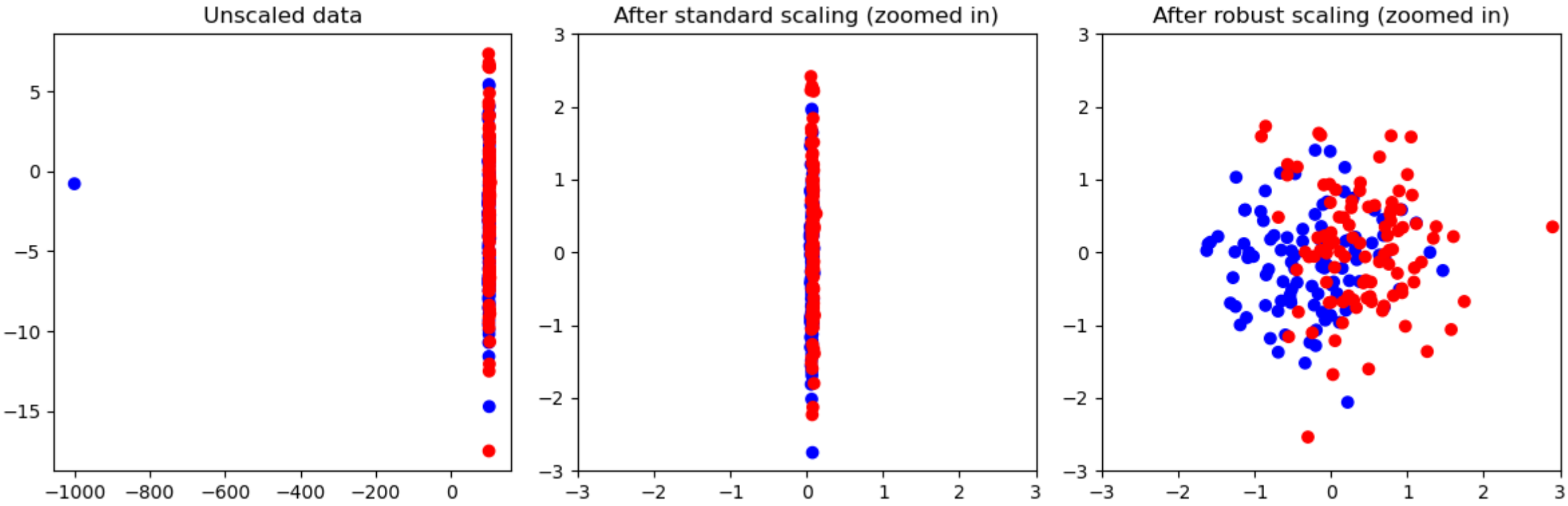


Scalers: Standart

Standart scaler: $x_{scaled} = \frac{x - x_{mean}}{x_{std}}$

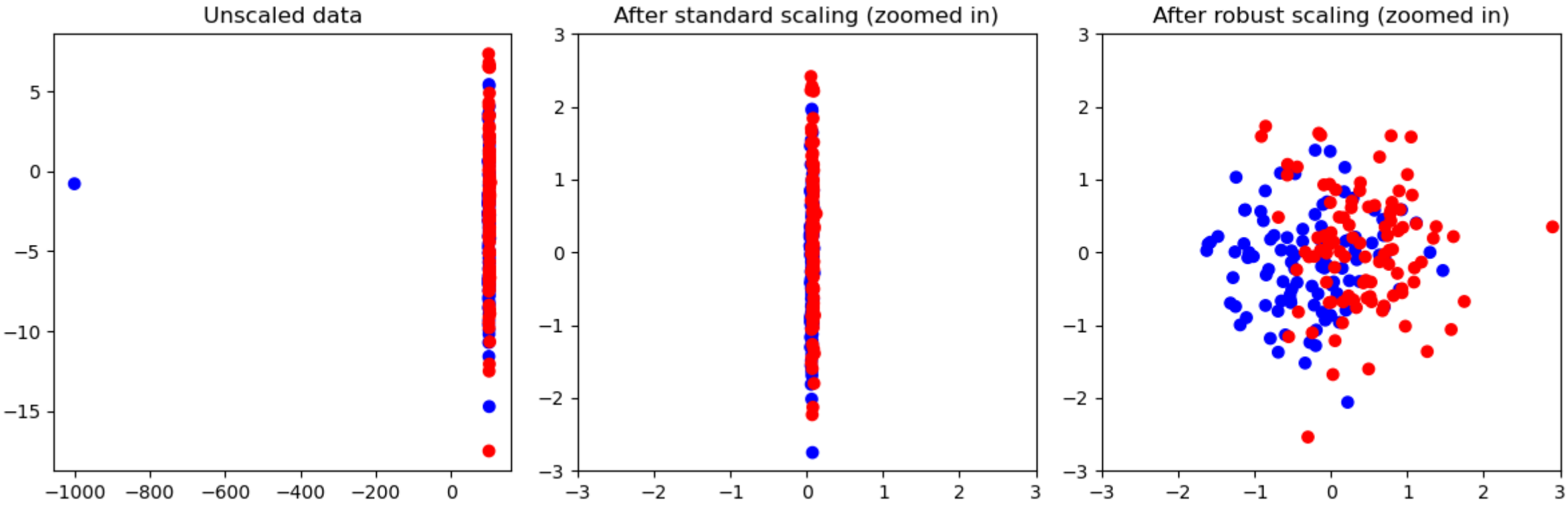


Scalers: Robust Scaler



Robust scaler: $x_{scaled} = \frac{x - x_{median}}{percentile_{max}(x) - percentile_{min}(x)}$

Scalers: Robust Scaler



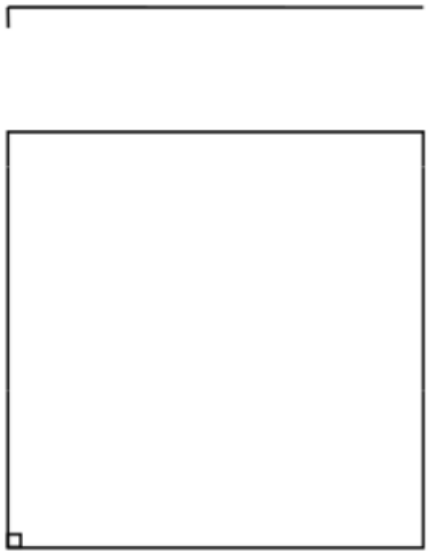
Robust scaler: $x_{scaled} = \frac{x - x_{median}}{percentile_{0.75}(x) - percentile_{0.25}(x)}$

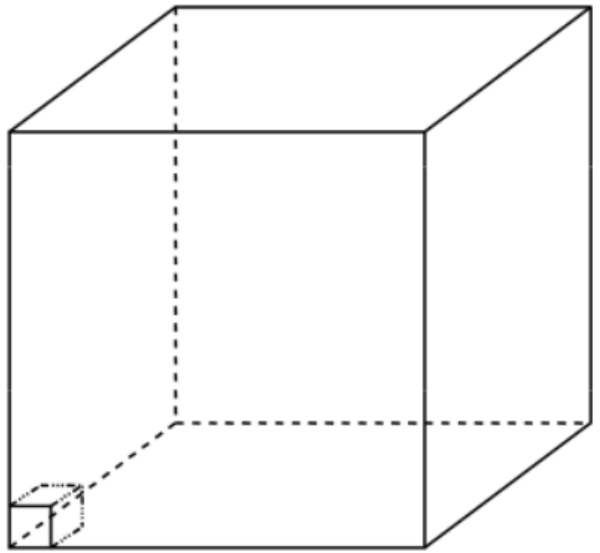
Curse of Dimensionality

kNN breaks down in high-dimensional space. “Neighborhood” becomes very large.

Assume 5000 point uniformly distributed in the unit hypercube and we want to apply 5-NN for point at the origin:

- 1-d: we must go a distance of 5/5000 = 0.01 on the average to capture 5 nearest neighbors
- 2-d we must go $\sqrt{0.001}$ to get a square that contains 0.001 of the volume
- In k dimensions we must go $\sqrt[k]{0.001}$





Example: Greedy Feature Selection in kNN

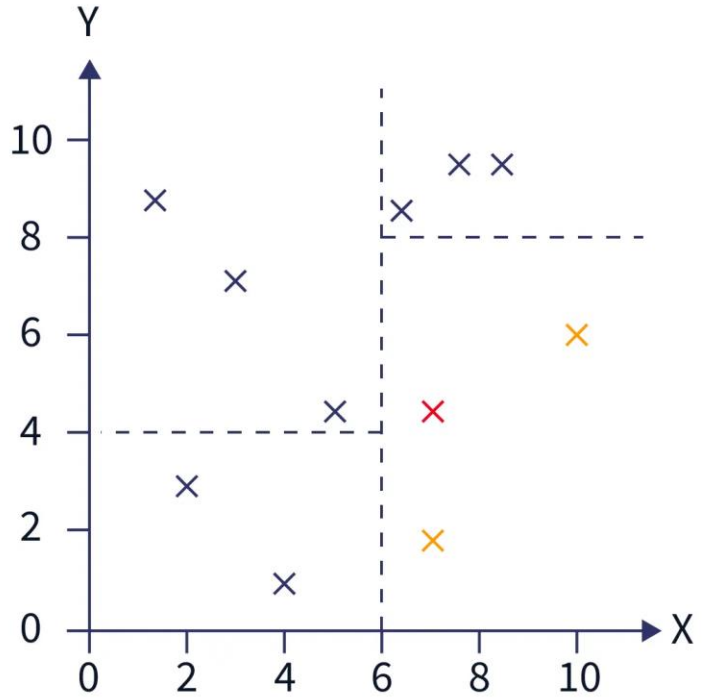
1. Normalize the features
2. Define set features F, initially empty
3. Select the best feature l_1 (by LOO) and add it to F
4. Select the best feature l_2 by union $F \cup \{l_2\}$ and add it to F
5. Repeat 4 while LOO or the validation error is decreasing (or the accuracy is increasing).

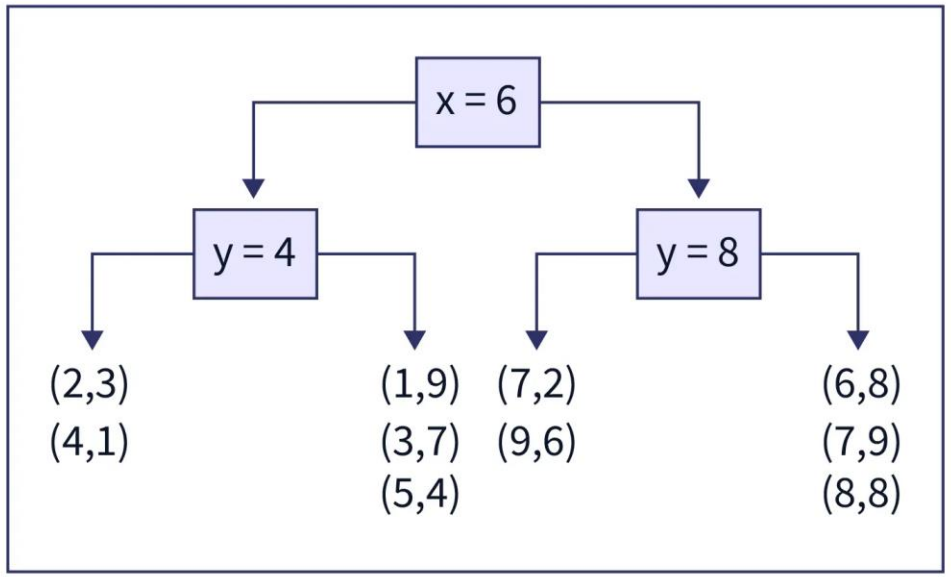
K-D Tree

Building a K-D tree from training data: pick dimension, find median, split data, repeat

Find NNs for new point (7, 4):

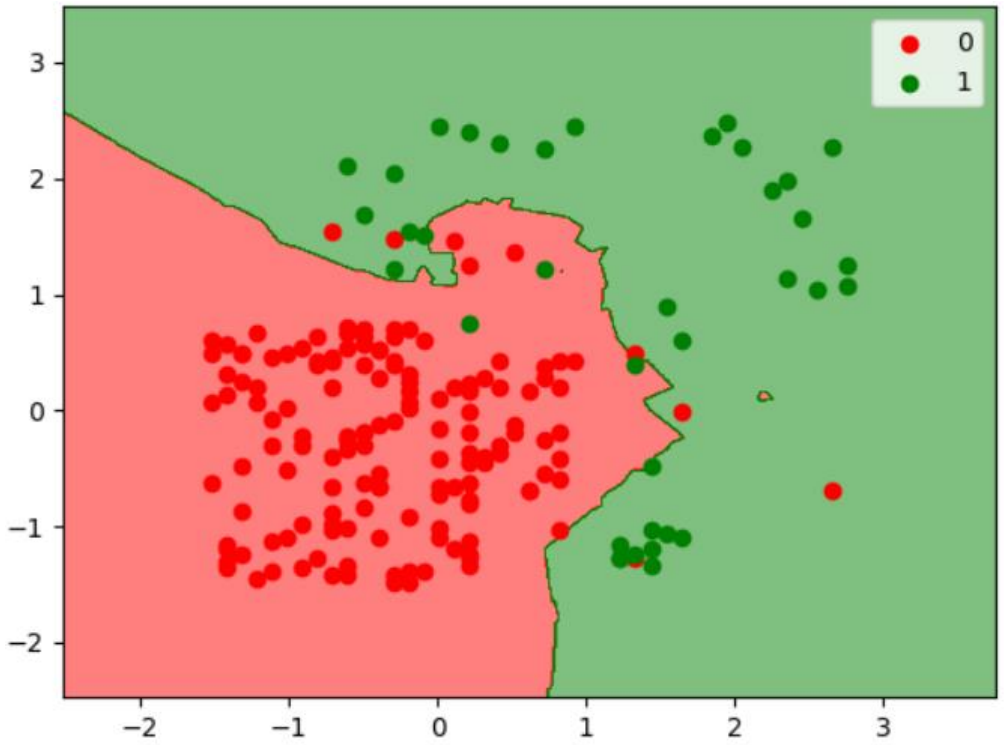
- Find region containing (7, 4)
- Compare to all points in region



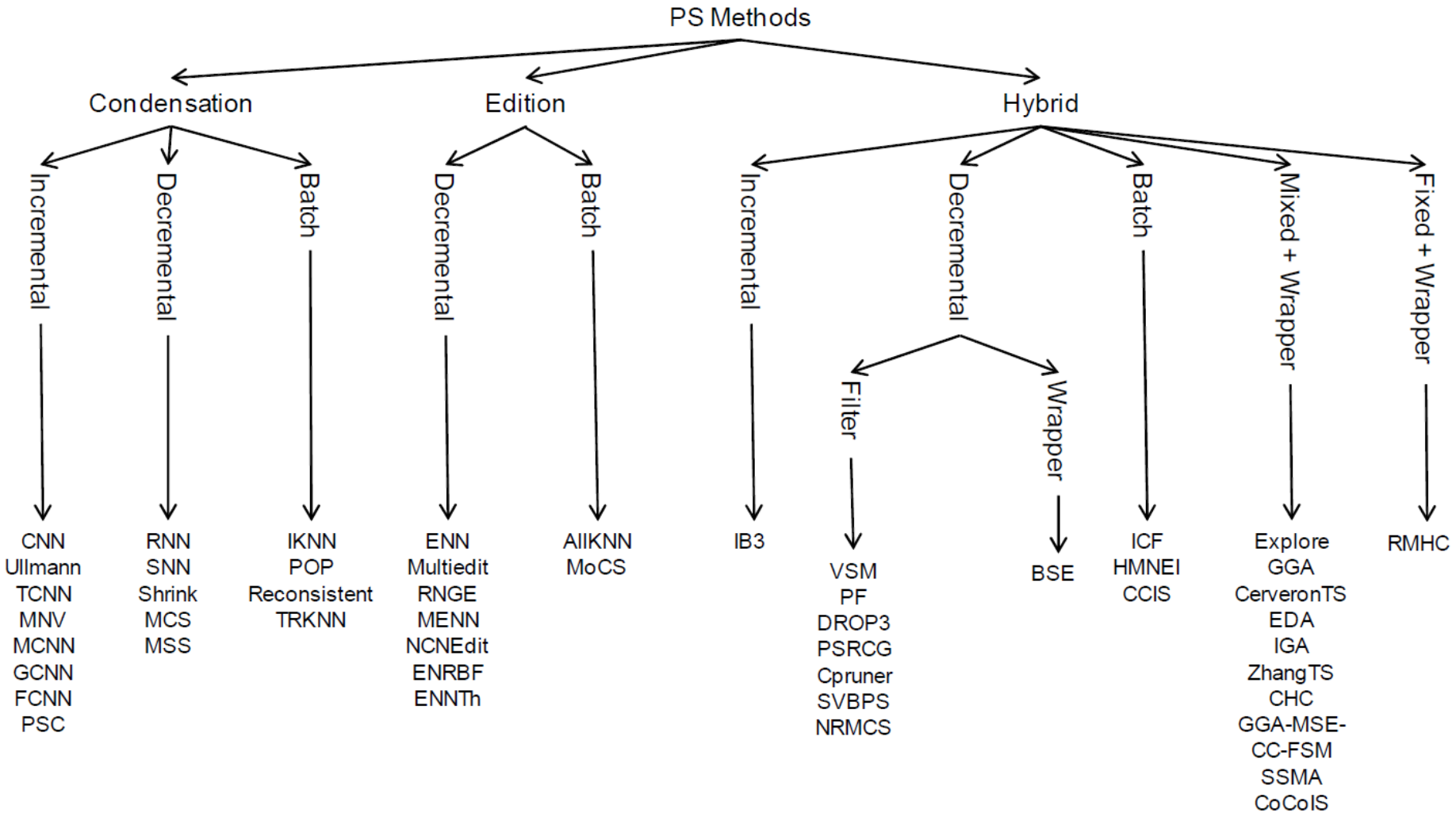


Prototype Selection

$$h(x;\Omega) = \arg\max_{y \in Y} \sum_{x_i \in \Omega} [y_i = y] w(x_i, x), \Omega \subseteq D$$



Prototype Selection Methods Taxonomy



Decremental Reduction Optimization Procedure: DROP5

- Start with the full dataset
- Sort data points by the affinity to the closest incorrect class
- Go in the ascending order
- Delete point x, if that does not increase the LOO error for the points that consider x one of their closest neighbors